Refactoring

Refactoring is the disciplined process of restructuring existing computer code, changing its internal structure to improve readability, maintainability, and design, **without altering its external behavior or functionality**.

It's like tidying up a messy room – the room still serves its purpose (external behavior), but it's easier to find things and use later (internal structure).

This is done through small, behavior-preserving transformations (e.g., renaming variables, extracting methods) to reduce technical debt and make the software simpler and more understandable.

Key Goals & Benefits

- **Improve Readability:** Make code clearer and easier for developers to understand.
- **Reduce Complexity:** Simplify convoluted logic and designs.
- **Enhance Maintainability:** Easier to fix bugs and add new features later.
- **Reduce Technical Debt:** Address issues from rushed code, like duplication or unclear names.

How It Works (The Process)

1. **Identify "Code Smells":** Look for problems like duplicated code, long methods, or unclear names.
2. **Perform Small Transformations:** Apply specific, tiny refactorings (e.g., "Extract Method," "Rename Variable").
3. **Test After Each Step:** Run automated tests to ensure no behavior has changed (keeping it "green").
4. **Keep Functionality Separate:** Don't add new features or fix bugs *during* the refactoring; do it before or after.

Common Refactoring Techniques

- **Extract Method:** Turn a code snippet into its own reusable function.
- **Rename:** Give clearer names to variables, methods, or classes.
- **Replace Temp with Query:** Turn temporary variables into methods.
- **Encapsulate Field:** Use getter/setter methods instead of direct field access.

In essence, refactoring is about paying down the "interest" on technical debt to make future development cheaper, faster, and less buggy.

Refactoring vs. Rewriting

- **Refactoring:** Small, incremental, behavior-preserving changes to improve internal structure.
- **Rewriting (or "Defactoring"):** A larger overhaul, often involving significant changes to behavior and functionality, which is riskier.
- Refactoring improves existing code's internal structure without changing its external behavior (like spring cleaning),
  while
  Rewriting rebuilds the code from scratch, potentially using new tech, discarding the old to fix fundamental flaws.
- Refactoring is low-risk, gradual, and reduces technical debt incrementally, ideal for maintainability; Rewriting is high-risk, time-consuming, and suited for outdated systems beyond repair, aiming for modernization or new capabilities.

Extract Method - a code refactoring technique

The **Extract Method** refactoring technique involves moving a self-contained block of code into a new, separate method (or function) and replacing the original code with a call to the new method.

The primary goal is to improve code readability, reduce duplication, and make the code more modular and maintainable.

Example (Java/C# pseudocode)

Consider a method that prints customer details and the amount they owe.

Before Refactoring (Long Method)	After Refactoring (Extract Method)
The `printOwing` method handles two different levels of abstraction: • printing a general banner (high level) and • printing specific details like name and amount (low level).	The code responsible for printing the specific details is moved to a new method, `printDetails(double outstanding)`, and the original method now calls this new, descriptive method. This makes the `printOwing` method cleaner and easier to understand.

```java
void printOwing() {
    printBanner();

    // Print details
    System.out.println("name: " + name);
    System.out.println("amount: " + getOutstanding());
}
```

```java
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}

void printDetails(double outstanding) {
    System.out.println("name: " + name);
    System.out.println("amount: " + outstanding);
}
```

"Rename" - a code refactoring technique

"Rename" is a fundamental **code refactoring** technique used to improve code readability and maintainability without changing its external behavior.

This is typically done using an Integrated Development Environment (IDE) feature that automatically updates all references to the renamed element across the codebase.

Example of Rename Refactoring

A common example is renaming a variable, method, or class to give it a more meaningful name.

Before Refactoring	After Refactoring
`public void calculateTotal() { ... }` (A generic name)	`public void computeTotalPrice() { ... }` (A more specific name)
`String abc;` (A non-descriptive variable)	`String firstName;` (A clear, descriptive variable)
`class MyClass { ... }` (A generic class name)	`class CustomerDataHandler { ... }` (A name reflecting its purpose)

"Replace Temp with Query" - a code refactoring technique

"**Replace Temp with Query**" is a code refactoring technique that improves readability and maintainability by converting a temporary variable holding the result of an expression into a method (or property).

Problem

You have a temporary variable that stores the result of a simple expression, and this variable is used in multiple places within a method. This can lead to longer methods and prevents the extraction of other code blocks because the temporary variable's scope is local.

Solution

Extract the expression into its own method (a "query" method) and replace all instances of the temporary variable with calls to this new method.

Example Code:

Before

```java
double calculateTotal() {
    double basePrice = quantity * itemPrice; // Temp variable
    if (basePrice > 1000) {
        return basePrice * 0.95;
    } else {
        return basePrice * 0.98;
    }
}
```

After

```java
double calculateTotal() {
    if (basePrice() > 1000) {
        return basePrice() * 0.95;
    } else {
        return basePrice() * 0.98;
    }
}

double basePrice() { // Query method
    return quantity * itemPrice;
}
```

Performance Concerns

A common concern is that calling a method multiple times might be less efficient than using a single temporary variable.

**Martin Fowler's advice is to prioritize clean code and worry about performance only if profiling shows it to be a problem.

Most compilers are efficient, and if performance *does* become an issue, it's easy to cache the result or revert the change later.

[***Martin Fowler is a British software developer, author and international public speaker on software development, specializing in object-oriented analysis and design, UML, patterns, and agile software development methodologies, including extreme programming.]*

"Encapsulate Field" - a code refactoring technique

"Encapsulate Field" is a refactoring technique where a public field is hidden by making it private and providing public **accessor methods** (getters and setters) to control access to the data.

This improves maintainability and allows for validation logic or future changes to the internal representation without affecting external classes.

Example Code – Java

The following example demonstrates how to refactor a `Person` class to encapsulate the `name` field.

Before Refactoring (Code Smell)	After Refactoring (Encapsulated Field)
The field is publicly accessible, allowing any external code to modify it arbitrarily:	The `name` field is made `private`, and access is provided through `public` getter and setter methods. This allows control over how the data is used (e.g., adding validation in the setter).

java

```java
public class Person {
    public String name;
    // other fields and methods...
}

// Usage in client code:
Person p = new Person();
p.name = "John Doe"; // Direct access and modification
```

java

```java
public class Person {
    private String name; // Field is now private

    public String getName() { // Public getter method
        return name;
    }

    public void setName(String name) { // Public setter method (ca
        if (name != null && !name.isEmpty()) {
            this.name = name;
        } else {
            // Handle invalid name case (e.g., throw an exception,
            System.out.println("Error: Name cannot be empty.");
        }
    }
    // other methods...
}

// Usage in client code (updated to use accessors):
Person p = new Person();
p.setName("John Doe"); // Access via setter
String personName = p.getName(); // Access via getter
```

How to Perform the Refactoring

The general steps for manually encapsulating a field are:

1. Make the field `private` (or `protected`).
2. Create public **getter** (read access) and **setter** (write access) methods for the field.
3. Find all existing direct usages of the field throughout the codebase and replace them with calls to the newly created getter or setter methods.